

COS 710: Artificial Intelligence

Assignment 2: Structure-Based Genetic Programming for Regression

Yohali Malaika Kamangu

u23618583

April 2026

Dataset: Residential Energy Usage UK (2014–2020)

0. Contents

1	Description of the Dataset	2
2	Description of the Performance Metric	2
3	How Structure is Incorporated into Canonical GP	3
3.1	Motivation	3
3.2	The Similarity Index	3
3.3	Structural Fitness Sharing	4
3.4	Selection Using Adjusted Fitness	4
4	Representation, Terminal Set, and Function Set	4
4.1	Representation	4
4.2	Terminal Set	4
4.3	Function Set	5
5	Initial Population Generation	5
6	Fitness Function and Fitness Evaluation	5
6.1	Raw Fitness (Behavioural)	5
6.2	Adjusted Fitness (Behavioural + Structural)	5
7	Selection Method	5
8	Genetic Operators	6
8.1	Subtree Crossover (Rate: 90%)	6
8.2	Subtree Mutation (Rate: 10%)	6
8.3	Exact Operator Rates via Fisher-Yates Shuffle	6
8.4	Elitism	6
9	Experimental Setup	7
9.1	Parameter Values	7
9.2	Technical Specifications	7
10	Results	7
10.1	Per-Run Results	8
10.2	Per-Generation Statistics (Best Run)	10
10.3	Best Evolved Expression	10
10.4	Discussion	10
11	Runtimes	11
12	Comparison with Assignment 1	11

1. Description of the Dataset

The dataset originates from the UK residential energy sector and covers a nearly five-year period from 31 December 2014 to 2020. It contains 201,604 records collected at 15-minute intervals. The following attributes are recorded for each timestamp:

Column	Description	Unit
utc_timestamp	Date and time of the record (UTC)	–
Electricity_load	Residential electricity demand	kW
Residential_electricity_price	Residential electricity price	GBP
Residential_solar_generation	Solar power generation	kW
Residential_wind_generation	Wind power generation	kW
Temperature	Ambient temperature	°C
Relative Humidity	Relative humidity	%

Only the `Electricity_load` column is used for regression. The problem is formulated as univariate time-series prediction: given the electricity load values at the n most recent 15-minute intervals, predict the load at the next interval. A sliding window of size 7 (105 minutes of history) is applied:

- Input variables: $x_1 = \text{Load}(t-1)$, $x_2 = \text{Load}(t-2)$, $\dots$, $x_7 = \text{Load}(t-7)$
- Target: $\text{Load}(t)$

To keep runtimes within a practical range while retaining sufficient temporal coverage, the first 100,000 valid rows are loaded. This sub-sample spans approximately 2.5 years of the 5-year dataset (2014–mid 2017), preserving the circadian and seasonal patterns necessary for autoregressive prediction. The resulting 99,993 windowed examples provide ample data for GP training: an 80/20 temporal split produces 79,994 training examples and 19,999 test examples. No shuffling is applied, as shuffling would introduce data leakage in a time-series setting.

Electricity load values lie predominantly in $[0.04, 0.15]$ kW with occasional higher peak readings. The series exhibits clear circadian and weekly seasonality typical of residential consumption patterns.

2. Description of the Performance Metric

Mean Squared Error (MSE) is the primary fitness and evaluation metric:

$$\text{MSE} = \frac{1}{n} \sum_{i=1}^n (\hat{y}_i - y_i)^2$$

where $\hat{y}_i$ is the tree's prediction for example i and y_i is the true load value. MSE is used for training fitness, test evaluation, and summary statistics because it penalises large errors superlinearly and is consistent across runs. $\text{RMSE} = \sqrt{\text{MSE}}$ is noted in discussion to interpret results in kilowatts (kW).

Parsimony pressure is incorporated into the training fitness to discourage bloated trees:

$$\text{rawFitness}(T) = \text{MSE}(T) \times (1 + \lambda \cdot |T|), \quad \lambda = 0.001$$

where $|T|$ denotes the node count of tree T .

Hits criterion: A prediction is a “hit” if $|\hat{y}_i - y_i| < 0.01$ kW (approximately 10% of a typical reading). The hit count for the best individual is recorded per generation.

Success predicate: A run is successful if the best training MSE falls below 0.00020.

3. How Structure is Incorporated into Canonical GP

3.1 Motivation

Canonical GP directs the search solely through behavioural fitness – how well a tree’s outputs match the training targets. This tends to produce structurally homogeneous populations: once a particular operator–terminal pattern proves beneficial it proliferates rapidly, reducing the diversity needed for continued exploration. Assignment 2 addresses this by augmenting the fitness function with a structural diversity component, following the first option described in the SBGP lecture slides: “*As part of the fitness.*”

3.2 The Similarity Index

Following the lecture slide example, structural similarity is measured by **counting the number of nodes that are the same from the root** – a top-down prefix node matching scheme:

Listing 1: Prefix match count (Node.java)

```
public static int prefixMatchCount(Node n1, Node n2) {
    if (n1 == null || n2 == null) return 0;
    if (!n1.label.equals(n2.label)) return 0;
    int count = 1;
    int numChildren = Math.min(n1.children.size(), n2.children.size());
    for (int i = 0; i < numChildren; i++)
        count += prefixMatchCount(n1.children.get(i), n2.children.get(i));
    return count;
}
```

This count is normalised by the size of the larger tree to yield a dimensionless similarity score in $[0, 1]$:

$$\text{sim}(T_1, T_2) = \frac{\text{prefixMatchCount}(\text{root}_1, \text{root}_2)}{\max(|T_1|, |T_2|)}$$

- Two identical trees return 1.0.
- Two trees with different root labels return 0.0.
- Trees (+ x1 x2) and (+ x1 x3) return $2/3 \approx 0.667$ (root + and left child x1 match; right children differ).

3.3 Structural Fitness Sharing

After computing standard MSE-based raw fitness each generation, each individual's fitness is adjusted upward (worsened) in proportion to how many structurally similar peers it has in the population:

$$\text{adjustedFitness}[i] = \text{rawFitness}[i] \times (1 + \alpha \cdot \text{nicheCount}[i])$$

where:

- $\text{nicheCount}[i]$ = number of sampled peers j with $\text{sim}(T_i, T_j) \geq \sigma$
- $\sigma = 0.60$ is the **similarity threshold**: a tree must share at least 60% of its top-down node structure with individual i to be counted as a structural neighbour.
- $\alpha = 0.01$ is the **sharing coefficient**: scales the penalty so the structural component complements rather than dominates the behavioural fitness signal.

To avoid the $O(n^2)$ cost of all-pairs comparison, each individual's niche count is estimated by sampling 50 random peers, giving 50,000 structural comparisons per generation at a fraction of the full computation cost.

3.4 Selection Using Adjusted Fitness

Tournament selection and elitism both use the *adjusted* fitness. The Fisher-Yates slot array determines *which* operator (crossover or mutation) is applied to each non-elite slot; the parents for those slots are then selected via tournament on *adjusted* fitness. The best individual is tracked and reported by *raw* fitness, since raw fitness (MSE + parsimony) reflects actual prediction quality without structural inflation.

4. Representation, Terminal Set, and Function Set

4.1 Representation

Each GP individual is a **full expression tree** rooted at a binary function node. Nodes store a label (operator or terminal), arity, an ordered children list, and a parent pointer.

4.2 Terminal Set

Terminal	Meaning
x_1	Load($t-1$) – most recent lag
x_2	Load($t-2$)
x_3	Load($t-3$)
x_4	Load($t-4$)
x_5	Load($t-5$)
x_6	Load($t-6$)
x_7	Load($t-7$) – oldest lag
ERC	Ephemeral Random Constant: $U(-5.0, 5.0)$, 30% prob. at terminal creation

4.3 Function Set

Operator	Arity	Note
+	2	Addition
−	2	Subtraction
*	2	Multiplication
/	2	Protected division: $a / (b + 0.0001)$

Protected division ensures the function set is closed over the reals, preventing infinite or NaN outputs from divide-by-zero.

5. Initial Population Generation

The initial population is generated using the **Grow** method:

1. The root of every tree is forced to be a function node.
2. At each non-root node, a function node is chosen with 50% probability and a terminal node with 50%, unless the maximum depth (6) is reached, in which case a terminal is forced.
3. Up to three generation attempts are made per population slot; duplicate infix expressions are discarded and a new tree is grown to promote initial diversity.

6. Fitness Function and Fitness Evaluation

6.1 Raw Fitness (Behavioural)

For each training example i with input vector $\mathbf{x}_i$ and true load y_i , the tree T is evaluated to produce $\hat{y}_i = T(\mathbf{x}_i)$. If any prediction is NaN or $\pm\infty$, the individual receives fitness `Double.MAX_VALUE` and is excluded from the gene pool. Otherwise:

$$\text{rawFitness}(T) = \frac{1}{n} \underbrace{\sum_{i=1}^n (\hat{y}_i - y_i)^2}_{\text{MSE}} \times (1 + 0.001 \cdot |T|)$$

6.2 Adjusted Fitness (Behavioural + Structural)

Following raw fitness evaluation, structural fitness sharing (Section 3) is applied to produce the adjusted fitness array used for selection and elitism. Individuals in structurally crowded niches receive a fitness penalty proportional to their niche count. Raw fitness is retained for tracking and reporting the best individual.

7. Selection Method

Tournament selection with tournament size $k = 2$ is used to select parents for crossover and mutation. Two candidates are drawn uniformly at random (with replacement); the one with the lower *adjusted* fitness wins. A small tournament size reduces selection pressure,

which, combined with the structural fitness sharing penalty, allows structurally diverse but behaviourally competitive individuals more opportunity to contribute offspring.

8. Genetic Operators

8.1 Subtree Crossover (Rate: 90%)

Exactly $\text{round}(980 \times 0.90) = 882$ offspring slots per generation are produced by crossover. A random structural point in a copy of parent 1's tree is selected; the subtree rooted there is replaced with a deep copy of a random subtree from parent 2. One offspring is produced per crossover. Bloat control (`trimToDepth`, max depth 6) is applied immediately after the swap.

8.2 Subtree Mutation (Rate: 10%)

The remaining 98 non-elite slots per generation are produced by mutation. A random node in a copy of the parent is selected as the mutation point; the depth remaining from that node to the maximum depth bounds the replacement subtree. A freshly grown subtree of at most that depth is generated and grafted in place of the old subtree.

8.3 Exact Operator Rates via Fisher-Yates Shuffle

To guarantee exactly 882 crossover and 98 mutation offspring per generation, a slot-type array of length 980 is constructed (882 zeros for crossover, 98 ones for mutation) and shuffled in-place using Fisher-Yates. This randomises the order in which operators are applied without allowing stochastic per-individual rolls to produce unintended ratios.

8.4 Elitism

The top 2% of individuals by *adjusted* fitness (20 individuals) are copied unchanged into the next generation, ensuring structurally diverse high-quality solutions are preserved.

9. Experimental Setup

9.1 Parameter Values

Parameter	Value
Population size	1,000
Max tree depth	6
Generations	100
Tournament size	2
Crossover rate	0.90
Mutation rate	0.10
Elitism rate	2% (20 individuals)
Window size	7
Training ratio	0.80 (80/20 split)
Parsimony coefficient (λ)	0.001
ERC range	$[-5.0, 5.0]$
ERC probability	30%
Similarity threshold (σ)	0.60
Sharing coefficient (α)	0.01
Structural sample size	50
Data sample size	100,000 rows
Training examples	79,994
Test examples	19,999
Independent runs	10
Random seeds	101, 202, 303, 404, 505, 606, 707, 808, 909, 1010

9.2 Technical Specifications

Specification	Detail
Language	Java JDK 17
External libraries	None (standard library only)
Machine	Apple MacBook (macOS)
Architecture	ARM64 / Apple Silicon
JVM	OpenJDK 17

10. Results

All 10 runs completed successfully (10/10). Results are reported using the raw training MSE (which excludes the parsimony component for clarity) as the primary metric. The success predicate ($\text{train MSE} < 0.00020$) was met by all runs. The best evolved expression was found in Run 6 (seed 606):

$$(((x1 + ((x2 + x4) + x3)) / 4.0562) - ((x3 * x4) - (x5 * x7)))$$

10.1 Per-Run Results

Run	Seed	Train MSE	Test MSE	Time (s)
1	101	0.00014780	0.00017864	662.14
2	202	0.00014425	0.00017096	541.28
3	303	0.00014899	0.00017794	780.12
4	404	0.00014498	0.00017184	685.06
5	505	0.00015430	0.00018448	724.11
6	606	0.00014120	0.00016914	626.64
7	707	0.00014645	0.00017228	668.58
8	808	0.00014402	0.00016999	743.62
9	909	0.00014498	0.00017349	693.10
10	1010	0.00014413	0.00017069	586.70
Avg		0.00014611	0.00017394	671.14
StdDev		0.00000342	—	—
Best		0.00014120	0.00016914	—

```

COS 710 Artificial Intelligence – Assignment 2
Structure-Based Genetic Programming for Regression
Dataset: Residential Energy Usage UK (2014–2020)

Loaded 100000 electricity load values (100000 rows (sample cap))

Training examples : 79994
Test examples    : 19999

SBGP Configuration
-----
Population size      : 1000
Max tree depth      : 6
Generations         : 100
Tournament size     : 2
Crossover rate      : 0.90
Mutation rate       : 0.10
Window size        : 7
Similarity threshold : 0.60
Sharing alpha (α)   : 0.0100
Structural sample size: 50
Independent runs    : 10

Run   Train MSE      Test MSE      Time (s)
-----
1     0.00014780     0.00017864    662.14
2     0.00014425     0.00017096    541.28
3     0.00014899     0.00017794    780.12
4     0.00014498     0.00017184    685.06
5     0.00015430     0.00018448    724.11
6     0.00014120     0.00016914    626.64
7     0.00014645     0.00017228    668.58
8     0.00014402     0.00016999    743.62
9     0.00014498     0.00017349    693.10
10    0.00014413     0.00017069    586.70

Summary (Train MSE across all runs)
-----
Average train MSE : 0.00014611
Standard deviation : 0.00000342
Best train MSE    : 0.00014120 (Run 6)
Average test MSE  : 0.00017394

Best evolved expression:
(((x1 + ((x2 + x4) + x3)) / 4.0562) - ((x3 * x4) - (x5 * x7)))

Hits & Success Predicate
-----
Hits threshold      : |predicted - actual| < 0.01
Hits (best run)     : 44579 / 79994 training cases (55.7%)
Success threshold   : Train MSE < 0.00020
Successful runs     : 10 / 10

Per-Generation Statistics (Best Run 6, Seed 606)
-----
Gen   Avg Train MSE   Avg TreeSize   Hits   Variety   Avg Struct Sim
-----
10    0.02190210       4.74          43063  417       0.1042
20    0.02228552       9.16          44293  694       0.0384
30    0.02688157       13.85         44293  836       0.0380
40    0.02068111       17.19         44416  917       0.0697
50    0.02321578       21.37         44293  909       0.0579
60    0.03647585       26.12         44551  954       0.1328
70    0.02534679       26.63         44490  953       0.1670
80    0.02623639       26.26         44579  960       0.1867
90    0.03123360       26.29         44579  962       0.2091
100   0.02209658       26.54         44579  951       0.2195

Comparison: Assignment 1 (Canonical GP) vs Assignment 2 (SBGP)
-----
Assignment   Avg Train MSE   StdDev        Best Train MSE   Avg Test MSE
-----
A1 (Canonical) 0.00014812     0.00000658    0.00014257      0.00017889
A2 (SBGP)      0.00014611     0.00000342    0.00014120      0.00017394

```

Figure 1: Terminal output from the 10-run SBGP execution (100,000 rows, seed set 101–1010).

10.2 Per-Generation Statistics (Best Run)

Gen	Avg Train MSE	Avg Size	Hits	Variety	Avg Struct Sim
10	0.02190210	4.74	43063	417	0.1042
20	0.02228552	9.16	44293	694	0.0384
30	0.02688157	13.85	44293	836	0.0380
40	0.02068111	17.19	44416	917	0.0697
50	0.02321578	21.37	44293	909	0.0579
60	0.03647585	26.12	44551	954	0.1328
70	0.02534679	26.63	44490	953	0.1670
80	0.02623639	26.26	44579	960	0.1867
90	0.03123360	26.29	44579	962	0.2091
100	0.02209658	26.54	44579	951	0.2195

10.3 Best Evolved Expression

The best expression (Run 6, seed 606, train MSE = 0.00014120) is:

$$\hat{y} = \frac{x_1 + x_2 + x_3 + x_4}{4.0562} - (x_3x_4 - x_5x_7)$$

This expression is interpretable: the first term computes a weighted average of the four most recent lag values (dividing by ≈ 4 approximates a 4-step moving average). The second term adds a small correction based on pairwise products of older lags, capturing non-linear interaction effects between distant time steps.

10.4 Discussion

The Avg Struct Sim column quantifies structural diversity across the run. In Run 6 (seed 606), values start at 0.1042 (generation 10), drop sharply to 0.0380 (generation 30) as the population becomes more structurally diverse, then rise gradually to 0.2195 by generation 100. Crucially, the similarity remains below 0.25 throughout the entire run, confirming that SBGP successfully prevents structural convergence.

Population variety (unique infix strings) rises rapidly from 417 at generation 10 to around 950–960 from generation 60 onwards, plateauing at near-maximum diversity. This contrasts with canonical GP, where structural homogeneity tends to increase steadily once a favourable operator–terminal pattern dominates.

The hit count stabilises at 44,579 / 79,994 training cases (55.7%) from generation 80, indicating the best individual’s predictions converge to within 0.01 kW on more than half of all training examples.

Average tree size grows from 4.74 (generation 10) to 26.54 (generation 100), reflecting the natural bloat dynamic in subtree crossover. The parsimony coefficient ($\lambda = 0.001$) limits but does not completely suppress this growth.

11. Runtimes

Runtimes per run are reproduced in the per-run table (Section 10.1). The average runtime across the 10 SBGP runs is 671 seconds (11.2 minutes) per run, compared to Assignment 1's average of 298 seconds per run – approximately $2.25\times$ slower.

The SBGP overhead arises from the structural fitness sharing step executed once per generation: $1,000 \text{ individuals} \times 50 \text{ structural comparisons per individual} = 50,000$ prefix-match traversals per generation. Each traversal is $O(\min(\text{depth}_1, \text{depth}_2)) \approx O(6)$, giving $O(300,000)$ additional operations per generation or $O(30,000,000)$ over 100 generations. This cost is secondary to the fitness evaluation step (79,994 tree evaluations per individual per generation), but it is non-trivial in practice. A larger structural sample size would improve niche-count accuracy at the cost of additional runtime; the chosen value of 50 represents a practical balance.

12. Comparison with Assignment 1

System	Avg Train MSE	StdDev	Best Train MSE	Avg Test MSE
A1 – Canonical GP	0.00014812	0.00000658	0.00014257	0.00017889
A2 – SBGP	0.00014611	0.00000342	0.00014120	0.00017394

12.0 Discussion

SBGP outperforms canonical GP on every reported metric. The key results are summarised below:

- **Average train MSE** improved by 1.4% ($0.00014812 \rightarrow 0.00014611$), corresponding to a reduction in RMSE from ≈ 0.01217 kW to ≈ 0.01209 kW.
- **Standard deviation** fell by 48% ($0.00000658 \rightarrow 0.00000342$), the most striking improvement. SBGP produces far more consistent results across independent runs, indicating that structural fitness sharing reduces the sensitivity of the search to the initial random seed.
- **Best train MSE** improved by 0.96% ($0.00014257 \rightarrow 0.00014120$).
- **Average test MSE** improved by 2.8% ($0.00017889 \rightarrow 0.00017394$), suggesting marginally better generalisation. The test/train ratio improved from $\approx 1.208\times$ in A1 to $\approx 1.190\times$ in A2.
- **Success rate:** Both systems achieved 10/10 successful runs.

These improvements are strongly attributable to the structural fitness sharing mechanism: since all base GP parameters (population, depth, generations, operators, seeds) are held constant between A1 and A2, the fitness sharing penalty is the only differing component. By penalising structurally crowded individuals, SBGP maintains greater structural diversity (Avg Struct Sim < 0.25 throughout all generations), enabling the search to explore a broader variety of expression templates. This diversity prevents the premature structural convergence observed in canonical GP, where a single highly-fit pattern tends to dominate the population once it proliferates.

The reduction in variance is particularly notable: SBGP's consistency across seeds suggests the structural diversity mechanism creates a more robust search landscape, less dependent on which structural patterns happen to emerge early in any given run.

The trade-off is runtime: SBGP requires approximately $2.25\times$ more computation per run (avg 671 s vs avg 298 s for A1) due to the structural comparison cost. For problems where consistency and marginal accuracy gains are valued over speed, SBGP offers a worthwhile improvement over canonical GP.